

Prof. Me. Hélio Esperidião

ORIENTAÇÃO A OBJETOS

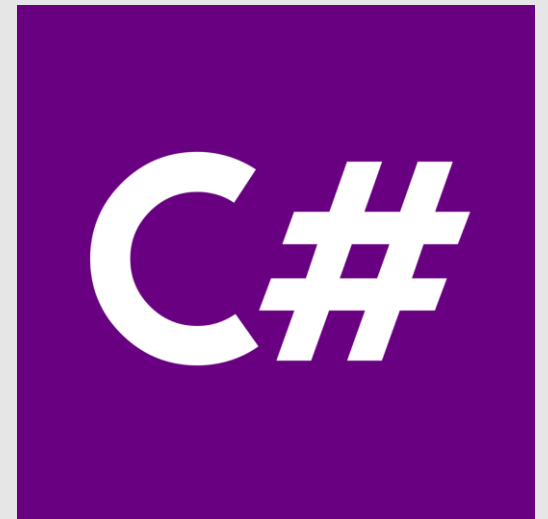


Programação orientada a objetos

- **Paradigma** significa **modelo, padrão**.
- POO É um **paradigma** que atualmente está entre os mais difundidos.
- Isso acontece porque se trata de um padrão que tem evoluído muito, principalmente em questões voltadas ao reaproveitamento de código, o que é muito importante no desenvolvimento de qualquer aplicação moderna.

POO

- A Programação Orientada a Objetos (POO) diz respeito a um padrão de desenvolvimento que é seguido por muitas linguagens, como C# e Java





Representações simplificadas

- Quando representamos elementos reais de forma simplificada e padronizada, estamos criando um modelo para esses elementos.

Modelos

- Modelos são representações simplificadas de objetos, pessoas, itens, tarefas, processos, conceitos, ideias etc., usados comumente por pessoas no seu dia a dia, independentemente do uso de computadores.

Modelos possuem

- Nome
 - Definição que representa o modelo
- Atributos
 - Propriedades e características do modelo
- Métodos
 - Ações realizadas pelo modelo.

Abstração



Abstração é o conceito de simplificar a complexidade, concentrando-se apenas nos aspectos essenciais de um objeto ou sistema, enquanto oculta os detalhes não essenciais.



No contexto da programação orientada a objetos, a abstração envolve a criação de classes e objetos que representam entidades do mundo real ou conceitos.

Interface de uma classe.

- A abstração permite que você modele objetos de acordo com suas características relevantes, ignorando os detalhes internos complexos que não são importantes para o uso geral.

Processo de Abstração

- A abstração é utilizada para criar as classes:
 - Definir o nome da classe
 - Definir as propriedades(Atributos) da classe
 - Definir as funções (métodos da classe)

Abstraindo...

- O processo de abstração é utilizado para definir o nome da classe, suas características e ações
- Imagine as característica de uma pessoa nos contextos:
 - Eleitor
 - Nome da classe?
 - Atributos?
 - Ações no sistema?
 - Aluno
 - Nome da classe?
 - Atributos?
 - Ações no sistema?
 - Motorista
 - Nome da classe?
 - Atributos?
 - Ações no sistema?

Características do Objeto:

Atributos descrevem as características das propriedades de um objeto, ou seja, as informações que o objeto contém.

Se estamos modelando um molde de "Carro", seus atributos poderiam incluir "cor", "marca", "modelo", "ano" etc.

Atributos

Os atributos, também conhecidos como campos, propriedades ou variáveis de instância, são as características ou dados que definem o estado de um objeto.

Representam informações que uma classe possui e podem variar de um objeto para outro da mesma classe.

Atributos são essenciais para modelar e representar objetos do mundo real de forma digital.

Variáveis de Instância

Os atributos são variáveis da instância.

Cada objeto criado a partir da classe terá suas próprias cópias dessas variáveis, representando o estado único do objeto.

Método

- Método vem a ser um grupo de instruções em uma classe que define como os objetos da classe se comportarão.
- Os métodos são análogos às funções em outras linguagens
- Também podem ser consideradas as ações.



Métodos

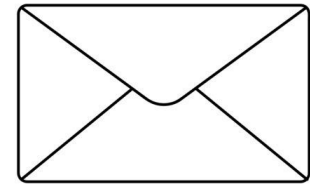
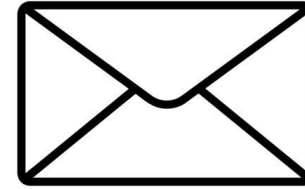
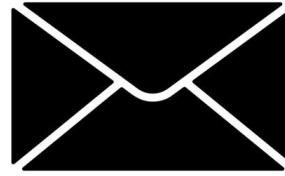
Um método é um bloco de código que define um conjunto de ações ou operações que um objeto pode realizar.

Ele representa o comportamento ou as ações associadas a um objeto. Os métodos são definidos nas classes e podem ser chamados para realizar tarefas específicas.

Um método pode receber argumentos (parâmetros) e pode retornar um valor após a conclusão da operação.

Os métodos encapsulam a lógica e as operações associadas a um objeto.

Mensagens



- Em programação orientada a objetos, a comunicação entre objetos é realizada por meio do envio de mensagens.
- **Uma mensagem é uma solicitação para que um objeto execute um determinado método.**
- Quando um objeto recebe uma mensagem, ele procura o método correspondente em sua classe e executa o código associado.
- As mensagens representam as interações entre objetos e são a base da comunicação em sistemas orientados a objetos.
 - Quando um objeto precisa realizar uma ação, outro objeto envia uma mensagem a ele, solicitando a execução de um método específico.

Exemplo de Troca de mensagens

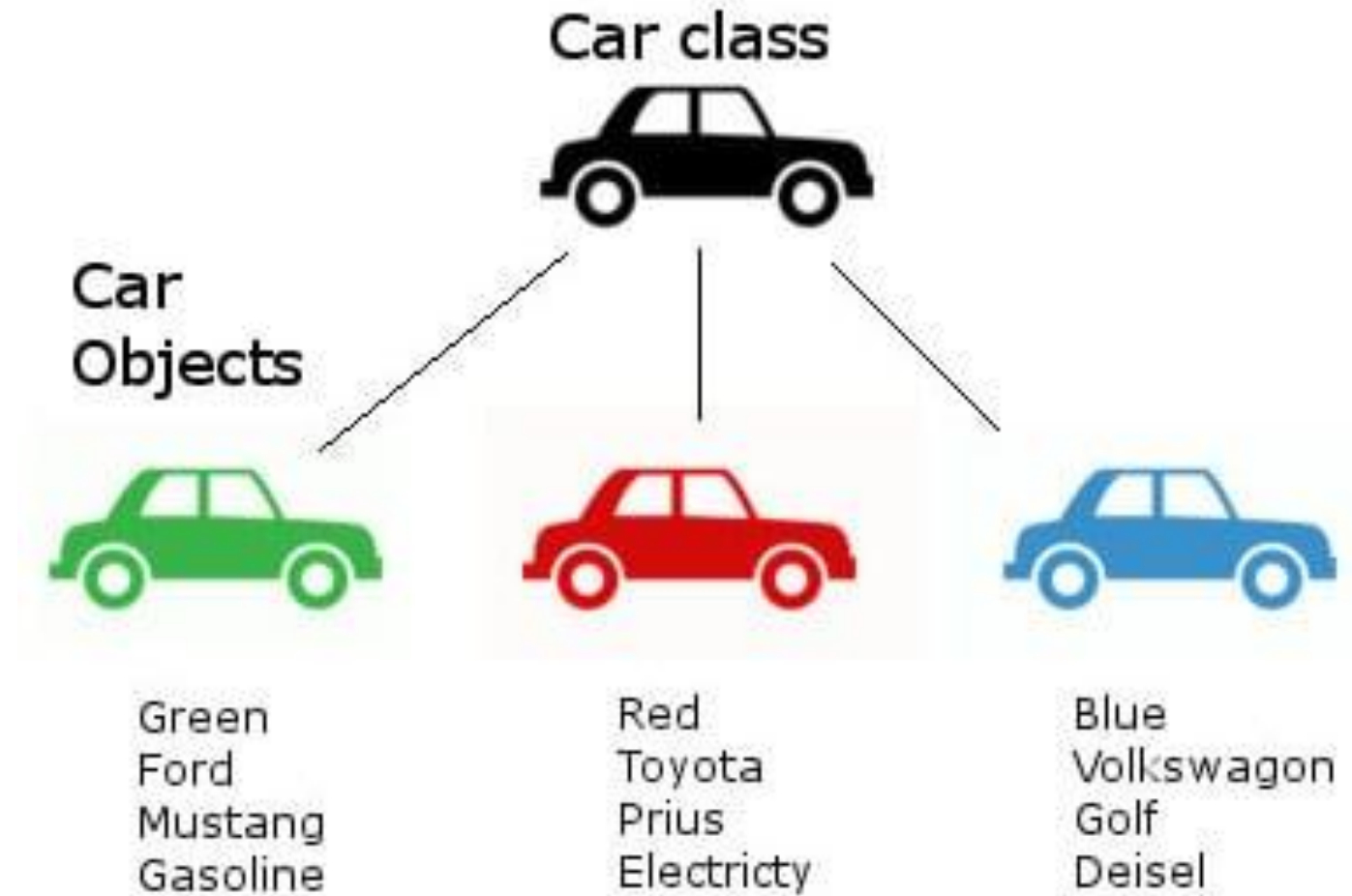
- Cenário
 - Temos um objeto ControleRemoto que envia mensagens para um objeto Televisao.
 - A interface pública de Televisao é formada pelos métodos ligar(), desligar() e alterarCanal().
 - Os atributos privados (canal, ligada) pertencem apenas à implementação interna.
- O que acontece:
 - ControleRemoto envia mensagens para Televisao (ex.: ligar(), alterarCanal()).
 - Televisao recebe essas mensagens, encontra os métodos correspondentes e executa a lógica.
 - Atributos privados (canal, ligada) não podem ser acessados diretamente pelo controle, apenas por meio da interface pública.

Objeto

- É uma instância de uma classe.
- Um objeto é capaz de armazenar estados através de seus atributos e reagir a mensagens enviadas a ele, assim como se relacionar e enviar mensagens a outros objetos.

Classe

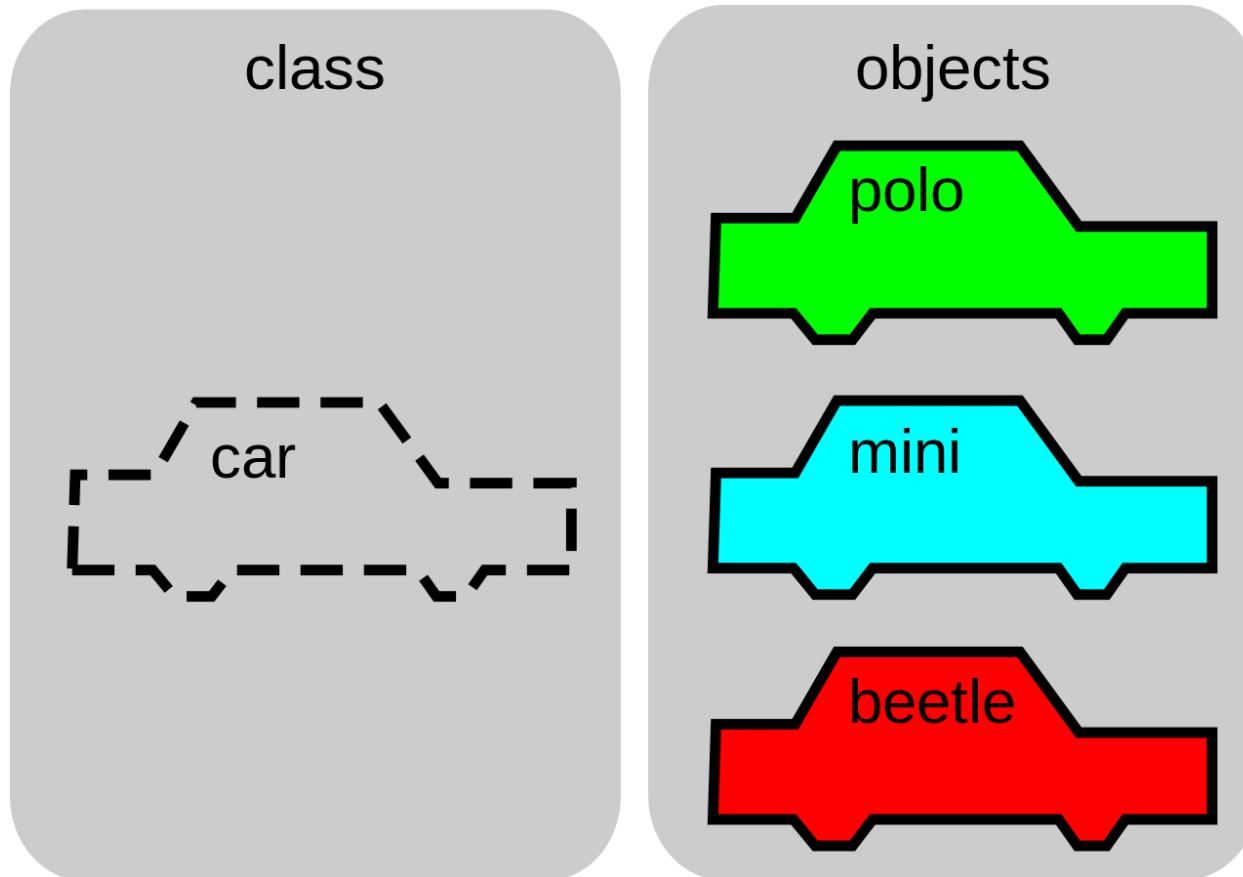
- É uma estrutura(Modelo) criada para dar origem ou modelar um objeto
- O exemplo mostra a estrutura básica de um carro.
- A estrutura pode ser modelada e dar origem a diversos carros



Instância (classe)

- As instâncias de uma classe compartilham o mesmo conjunto de atributos, embora sejam diferentes quanto ao conteúdo desses atributos.
- Por exemplo, a classe "**Animal**" descreve os atributos comuns a todas as instâncias da classe "**Animal**".
- Os objetos dessa classe podem ser semelhantes, mas variam em atributos tais como "**nome**" e "**tamanho**". A descrição da classe contém os itens correspondentes a esses atributos e define as operações ou ações relevantes para a classe, tais como "aumento de salário" ou "mudança do número de telefone".
- Pode-se então falar sobre uma instância com o nome = "Ted" e outra com o nome = "Bob".

Instância (classe)



Instâncias de classes e posições de memória

- Estão diretamente relacionadas, porque **cada vez que você cria um objeto (instância)** em um programa, ele é alocado em um endereço específico na memória do computador.

Conceito	Explicação
Classe	É um molde ou plano para criar objetos (não ocupa memória até ser instanciada).
Instância	É um objeto real criado a partir de uma classe, com seus próprios valores de atributos.
Posição de memória	É o endereço onde a instância é armazenada na memória RAM. Cada objeto tem um endereço único (referência).

Classe vs Objeto

- A classe é uma estrutura abstrata que define as características e os comportamentos que os objetos de um certo tipo(Classe) terão.
- Ela é uma espécie de "plano" para a criação de objetos.
- Por outro lado, um objeto é uma instância específica criada a partir de uma classe, com valores concretos para seus atributos e a capacidade de executar os métodos definidos na classe.
- A classe é a abstração, enquanto o objeto é a concretização dessa abstração.



Nomenclaturas – Java/POO

Nome da classe:

- Sempre inicia com a primeira letra em maiúscula.
 - **Pessoa**
- Caso o nome da classe seja composto por duas palavras ou mais, cada palavra deve iniciar com letra maiúscula.
 - **ClienteEspecial**
 - **CarroEsportivo**.

Atributos:

- Sempre inicia com letra minúscula.
- Caso o nome do atributo seja composto por mais de uma palavra, da segunda palavra a diante a primeira letra é maiúscula.
 - **nomeCliente**
 - **corQuadrado**
 - **nomeCompleto**

Nomenclatura – Java/POO

Métodos

- Sempre iniciam com letra minúscula.
- Caso o nome do atributo seja composto por mais de uma palavra, da segunda palavra a diante a primeira letra é maiúscula
 - **calcularArea()**
 - **cadastrarAluno()**
 - **cadastrarClienteEspecial()**

UML – Unified Modeling Language



- É uma linguagem para especificação, construção, visualização e documentação de sistemas de software;
- É a união da sintaxe gráfica de vários métodos, com vários símbolos removidos e vários adicionados;

UML



A Linguagem de Modelagem Unificada (UML) oferece uma abordagem versátil para a criação de modelos de sistemas.



Entre as várias formas de representação, um dos formatos mais proeminentes é o **diagrama de classes**.



São categorizados como diagramas de estrutura, pois delineiam os elementos essenciais que devem compor o sistema em análise.

Representação

- A representação da classe em si é configurada por um retângulo contendo três segmentos distintos.
- A seção superior abriga o nome da classe, o segmento intermediário abrange os atributos pertinentes à classe e o segmento inferior expõe os métodos ou operações acessíveis pela classe.

Classe quadrado

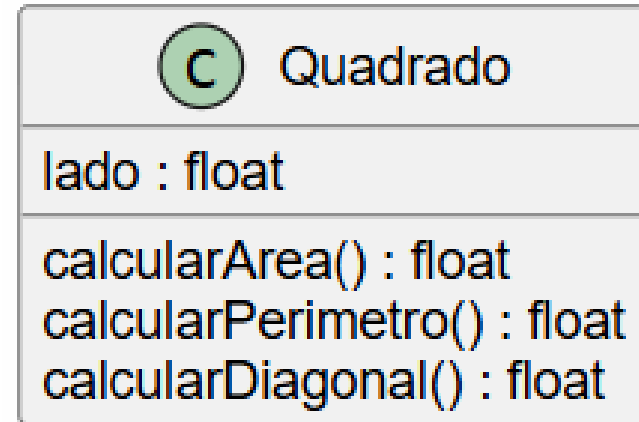
- Atributos: lado
- Métodos: **calcularPerimetro()**, **calcularArea()**, **calcularDiagonal()**
- Objetos: A classe Quadrado pode dar origem a vários quadrados diferentes.
- Instanciar: para dar origem a triângulos é necessário realizar a instância da classe Triângulo.
- A instância da origem a objetos.

Quadrado
lado: float
calcularArea(): float calcularDiagonal(): float calcularPerimetro(): float

PlantUML

```
@startuml
class Quadrado {
    lado : float

    calcularArea() : float
    calcularPerimetro() : float
    calcularDiagonal() : float
}
@enduml
```



Use para gerar uml: <https://plantuml.com/>

Quadrado.java

```
// Declaração da classe "Quadrado". Ela representa o conceito de um quadrado.
class Quadrado {
    // Atributo da classe: armazena o tamanho do lado do quadrado.
    // É do tipo float para permitir valores decimais.
    float lado;

    // Método público que calcula a área do quadrado.
    float calcularArea() {
        return lado * lado;
    }

    float calcularPerimetro() {
        return lado * 4;
    }

    float calcularDiagonal() {
        float raizDois = (float) Math.sqrt(lado * lado + lado * lado);
        return (float) raizDois;
    }
}
```

 Quadrado

lado : float

calcularArea() : float
calcularPerimetro() : float
calcularDiagonal() : float

Método main

- O método **main** é um ponto de entrada em um programa java.
- Ele é o método que é executado quando você inicia a execução de um programa Java.
- O método main atua como o ponto de partida da execução, a partir do qual o restante do código é acionado.

```
public static void main(String[] args) {  
    // Corpo do método main  
}
```

Método main

- **public**: O método main deve ser público, permitindo que ele seja acessível fora da classe.
- **static**: O método main deve ser estático para que possa ser chamado sem a necessidade de criar uma instância da classe.
 - Tudo que é estático não precisa de instância para ser utilizado.
- **void**: O método main não retorna nenhum valor, portanto, possui um tipo de retorno void.
- **String[] args**: O parâmetro args é um array de strings que pode ser usado para passar argumentos da linha de comando para o programa. Esses argumentos são úteis para fornecer entrada externa para o programa.

```
import java.util.Scanner;

public class QuadradoMain {
    public static void main(String[] args) {
        // Cria um objeto Scanner para capturar a entrada do usuário
        Scanner entrada = new Scanner(System.in);

        // Cria uma instância da classe Quadrado
        // q é um objeto da classe Quadrado
        // q possui uma posição de memória única
        Quadrado q = new Quadrado();

        System.out.print("Digite o valor do lado do quadrado: ");
        q.lado = entrada.nextFloat();

        System.out.println("Área: " + q.calcularArea());
        System.out.println("Perímetro: " + q.calcularPerimetro());

        System.out.println("Diagonal: " + q.calcularDiagonal());

        entrada.close();
    }
}
```

Várias instâncias(objetos) da classe Quadrado

```
import java.util.Scanner;

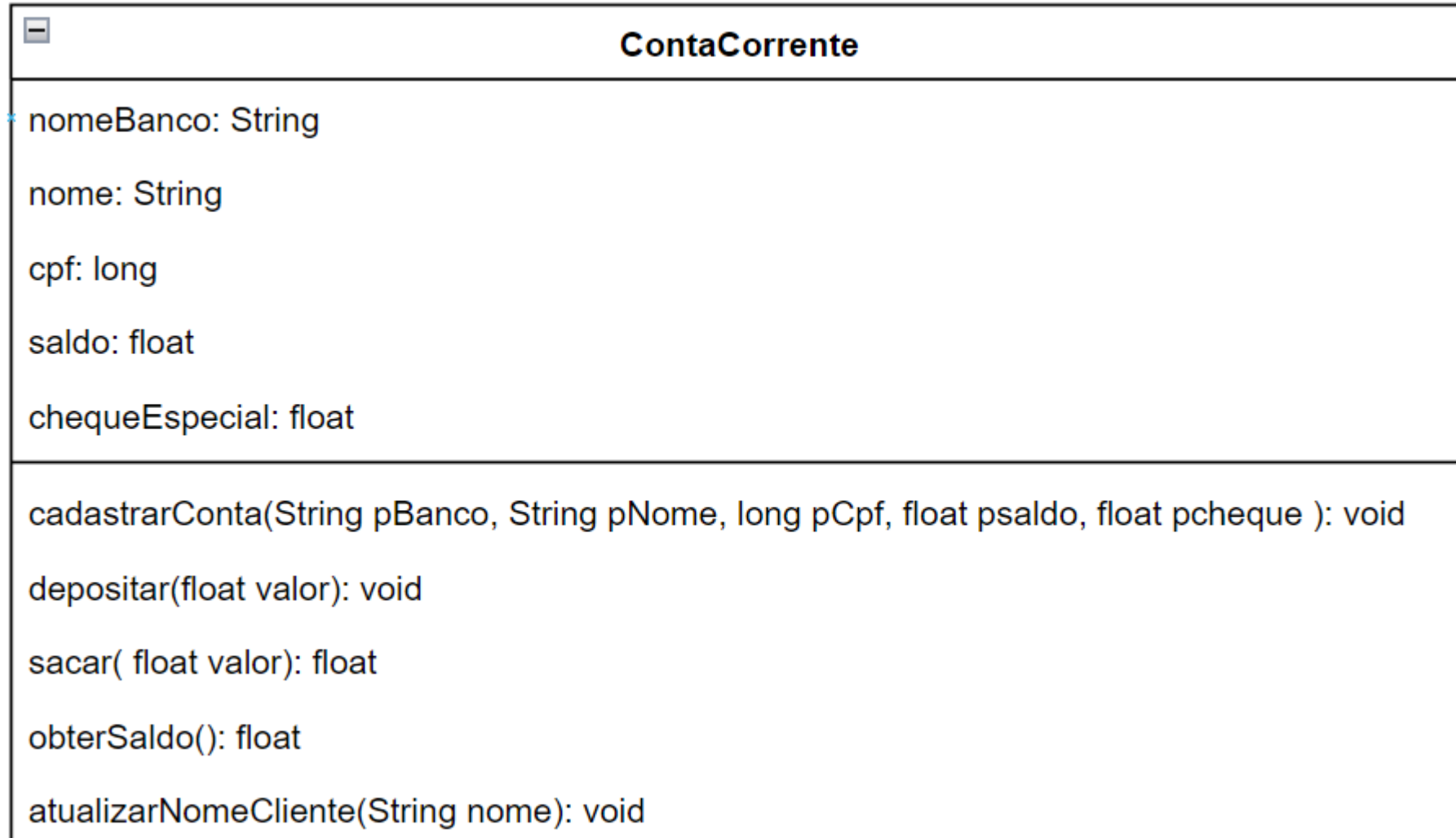
public class QuadradoMain2 {
    public static void main(String[] args) {
        Quadrado q1 = new Quadrado();
        Quadrado q2 = new Quadrado();
        Quadrado q3 = new Quadrado();
        q1.lado = 6;
        q2.lado = 9;
        q3.lado = 7;
        System.out.println(q1.calcularArea());
        System.out.println(q2.calcularArea());
        System.out.println(q3.calcularArea());
    }
}
```

q1, q2 e q3 são objetos diferentes e possuem posições de memória diferentes

Vetor de quadrados

```
import java.util.Scanner;
public class QuadradoMain3 {
    public static void main(String[] args) {
        // Cria um vetor de 3 posições para armazenar objetos da classe Quadrado
        Quadrado[] quadrados = new Quadrado[3];
        // Cria um objeto Scanner para ler dados do teclado
        Scanner entradaDados = new Scanner(System.in);
        // Armazena a quantidade de quadrados a serem processados
        int qtd = quadrados.length;
        for (int i = 0; i < qtd; i++) {
            System.out.printf("Lado do %dº quadrado = ", i + 1);
            // Cria uma nova instância de Quadrado
            quadrados[i] = new Quadrado();
            // Lê o valor do lado digitado pelo usuário e armazena no objeto Quadrado
            quadrados[i].lado = entradaDados.nextFloat();
        }
        float areaTotal = 0;
        // Loop para calcular a área total dos quadrados
        for (int i = 0; i < qtd; i++) {
            // Soma a área de cada quadrado à área total
            areaTotal += quadrados[i].calcularArea();
        }
        System.out.printf("Area total: %.2f", areaTotal);
        entradaDados.close();
    }
}
```

ContaCorrente



puml

@startuml

class ContaCorrente {

nomeBanco : String

nome : String

cpf : long

saldo : float

chequeEspecial : float

cadastrarConta(pBanco : String, pNome : String, pCpf : long, pSaldo : float, pCheque : float) : void

depositar(valor : float) : void

obterSaldo() : float

}

@enduml

 ContaCorrente

nomeBanco : String

nome : String

cpf : long

saldo : float

chequeEspecial : float

cadastrarConta(pBanco : String, pNome : String, pCpf : long, pSaldo : float, pCheque : float) : void

depositar(valor : float) : void

obterSaldo() : float

```

class ContaCorrente {
    String nomeBanco;
    String nome;
    long cpf;
    float saldo;
    float chequeEspecial;
    void cadastrarConta(String pBanco, String pNome, long pCpf, float pSaldo, float pCheque) {
        nomeBanco = pBanco;
        nome = pNome;
        cpf = pCpf;
        saldo = pSaldo;
        chequeEspecial = pCheque;
    }

    void depositar(float valor) {
        saldo = saldo + valor;
    }

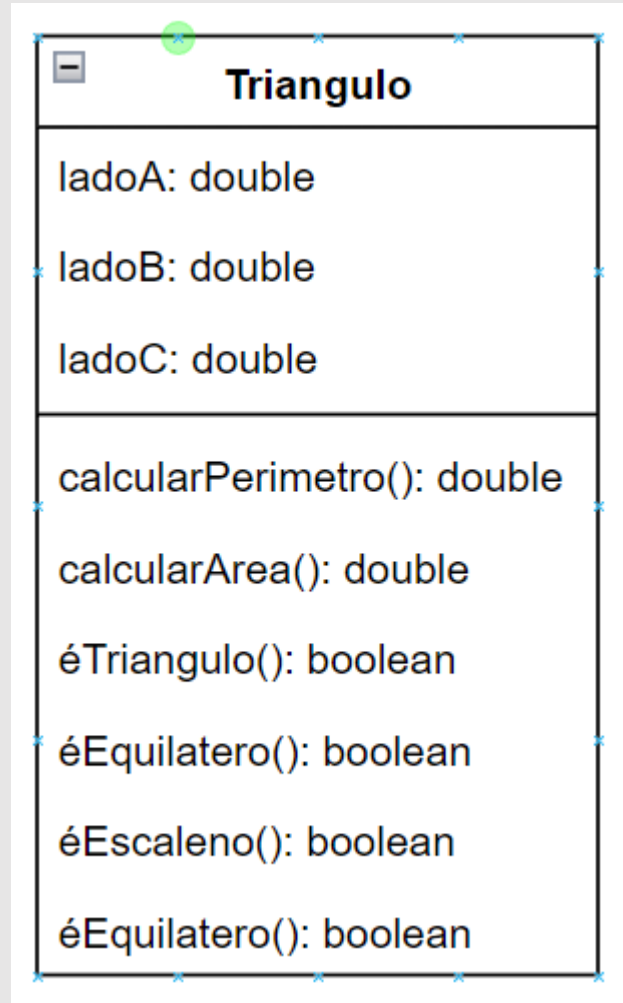
    float obterSaldo() {
        return saldo;
    }
}

```

ContaCorrente
nomeBanco: String
nome: String
cpf: long
saldo: float
chequeEspecial: float
cadastrarConta(String pBanco, String pNome, long pCpf, float psaldo, float pcheque): void
depositar(float valor): void
sacar(float valor): float
obterSaldo(): float
atualizarNomeCliente(String nome): void

```
public class ContaCorrenteMain {  
    public static void main(String[] args) {  
        ContaCorrente conta1 = new ContaCorrente();  
        conta1.cadastrarConta("banco Bom", "Ana", 11122233310L, 30, 0);  
        conta1.depositar(20);  
        System.out.println(conta1.obterSaldo());  
    }  
}
```

Classe triangulo



- Atributos: ladoA, ladoB, ladoC.
- Métodos: `éequilatero()`, `calcularPerimetro()` etc...
- Objetos: a classe triangulo pode dar origem a vários triângulos diferentes.
- Instanciar: para dar origem a triângulos é necessário realizar a instância da classe Triangulo. O ato de instanciar da origem a objetos.

Compilando java via console

- Compilar programa:
 - `javac Quadrado.java`
- Executar programa
 - `java Quadrado`

Recebendo dados via args

```
class Exemplo{  
    public static void main(String[] args) {  
        int qtd = args.length;  
        for(int i=0;i<qtd;i++){  
            System.out.printf("args[%d] = %s\n",i,args[i]);  
        }  
    }  
}
```

```
PS C:\temp> javac Exemplo.java  
PS C:\temp> java Exemplo dado1 dado2 dado3 dadoN  
args[0] = dado1  
args[1] = dado2  
args[2] = dado3  
args[3] = dadoN  
PS C:\temp> 
```

Onde construir os diagramas

- Diagramas: <https://app.diagrams.net/>
- IDE Online:
 - <https://www.jdoodle.com/online-java-compiler/>
 - <https://www.onlinegdb.com/>